FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	REYES, SOFIA BEATRICE ALEXA R.
Section:	AN42
Professor:	CRISOLA TAN
Date:	OCTOBER 29, 2026
GITHUB LINK	https://github.com/brryes/Reyes-SofiaBeatriceAlexa-CS0065-AN42.git

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.

6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking

✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation

✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft

- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

CHOSEN CASE STUDY: Case Study 1 – Disaster Response Decision Support System

1. Description of Your Case Base

The case base contains five sample disaster cases with different disaster types, severity levels, locations, and affected populations. Each case includes a response plan covering evacuation, shelter preparation, and resource allocation.

2. Explanation of Similarity Method

The program uses weighted similarity scoring based on four attributes: disaster type (35%), severity (30%), location (20%), and affected population (15%). Matching attributes receive higher scores, while population similarity depends on the difference between the affected populations. The case with the highest overall score is retrieved.

3. Adaptation Logic

The program adjusts the retrieved response plan according to the new disaster's severity, location, type, and affected population. It generates recommendations for evacuation, shelter preparation, and resource allocation suited to the given scenario.

SCREENSHOTS:

```
python -u "/Users/sofiabeatrice/Documents/intelligent sys/disaster_response_cbr.py"
sofiabeatrice@Sofias-MacBook-Air intelligent sys % python -u "/Users/sofiabeatri
ce/Documents/intelligent sys/disaster_re
sponse_cbr.py"
================================================================
 DISASTER RESPONSE DECISION SUPPORT SYSTEM (CBR)
================================================================
Current case base: 5 case(s)

=== EXISTING CASE BASE ===

Case ID: 1 | Name: Typhoon Ursula - Sample Case
  Disaster: Typhoon | Severity: High | Location: Coastal | Population: 3,000
  Evacuation: Evacuate residents in high-risk coastal and flood-prone zones.
  Shelter: Prepare temporary shelters and confirm safe capacity.
  Resources: Prioritize rescue boats, food, drinking water, and first-aid supplies.

Case ID: 2 | Name: Urban Flooding - Sample Case
  Disaster: Flood | Severity: High | Location: Urban | Population: 5,000
  Evacuation: Move residents from deeply flooded streets and low-lying neighborhoods.
  Shelter: Open accessible evacuation centers and check their remaining capacity.
  Resources: Prioritize rescue vehicles, clean water, food packs, and medical supplies.

Case ID: 3 | Name: Rural River Flood - Sample Case
  Disaster: Flood | Severity: Moderate | Location: Rural | Population: 1,200
  Evacuation: Move households near the river to designated safe areas when advised by authorities.
  Shelter: Prepare a nearby community shelter and transport assistance.
  Resources: Prepare drinking water, food packs, first-aid kits, and transport support.

Case ID: 4 | Name: Coastal Storm Surge - Sample Case
  Disaster: Typhoon | Severity: Moderate | Location: Coastal | Population: 2,200
  Evacuation: Issue early warnings and move residents away from storm-surge risk areas.
  Shelter: Prepare inland evacuation centers and transportation.
  Resources: Prepare communication equipment, food, water, and first-aid supplies.

Case ID: 5 | Name: Low-Severity Urban Flood - Sample Case
  Disaster: Flood | Severity: Low | Location: Urban | Population: 800
  Evacuation: Monitor flood-prone streets and advise residents to move if conditions worsen.
  Shelter: Identify a nearby safe location and keep evacuation centers on standby.
  Resources: Prepare warning messages, drinking water, basic first aid, and transport if needed.
```

```
=== ENTER NEW DISASTER ===
Case name (New Urban Typhoon
Disaster type (earthquake/flood/typhoon): typhoon
Flood/impact severity (high/low/moderate): high
Location type (coastal/rural/urban): urban
Estimated affected population: 5000

=== SIMILARITY ASSESSMENT ===
1. Typhoon Ursula - Sample Case - Similarity: 74.00%
2. Urban Flooding - Sample Case - Similarity: 65.00%
3. Coastal Storm Surge - Sample Case - Similarity: 41.60%
4. Low-Severity Urban Flood - Sample Case - Similarity: 22.40%
5. Rural River Flood - Sample Case - Similarity: 3.60%

=== RETRIEVED MOST SIMILAR CASE ===

Case ID: 1 | Name: Typhoon Ursula - Sample Case
  Disaster: Typhoon | Severity: High | Location: Coastal | Population: 3,000
  Evacuation: Evacuate residents in high-risk coastal and flood-prone zones.
  Shelter: Prepare temporary shelters and confirm safe capacity.
  Resources: Prioritize rescue boats, food, drinking water, and first-aid supplies.
Similarity score: 74.00%

=== ADAPTED RESPONSE PLAN ===
Adapted from: Typhoon Ursula - Sample Case
1. Evacuation: Prioritize urgent evacuation from high-risk areas. Coordinate with local emergency o
ing streets, drainage bottlenecks, and densely populated areas.
2. Shelter: Urgently confirm sufficient shelter capacity for up to 5,000 people, including accessib
3. Resources: Prioritize rescue teams, safe transport, drinking water, food, first-aid supplies, an
nd advisories.

=== LEARNING / RETENTION ===
New case saved successfully!
Saved to: /Users/sofiabeatrice/Documents/intelligent sys/disaster_cases.json
Updated case base contains 6 case(s).
```

```
Case ID: 1 | Name: Typhoon Ursula - Sample Case
  Disaster: Typhoon | Severity: High | Location: Coastal | Population: 3,000
  Evacuation: Evacuate residents in high-risk coastal and flood-prone zones.
  Shelter: Prepare temporary shelters and confirm safe capacity.
  Resources: Prioritize rescue boats, food, drinking water, and first-aid supplies.

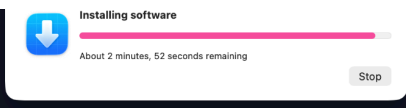
Case ID: 2 | Name: Urban Flooding - Sample Case
  Disaster: Flood | Severity: High | Location: Urban | Population: 5,000
  Evacuation: Move residents from deeply flooded streets and low-lying neighborhoods.
  Shelter: Open accessible evacuation centers and check their remaining capacity.
  Resources: Prioritize rescue vehicles, clean water, food packs, and medical supplies.

Case ID: 3 | Name: Rural River Flood - Sample Case
  Disaster: Flood | Severity: Moderate | Location: Rural | Population: 1,200
  Evacuation: Move households near the river to designated safe areas when advised by authorities.
  Shelter: Prepare a nearby community shelter and transport assistance.
  Resources: Prepare drinking water, food packs, first-aid kits, and transport support.

Case ID: 4 | Name: Coastal Storm Surge - Sample Case
  Disaster: Typhoon | Severity: Moderate | Location: Coastal | Population: 2,200
  Evacuation: Issue early warnings and move residents away from storm-surge risk areas.
  Shelter: Prepare inland evacuation centers and transportation.
  Resources: Prepare communication equipment, food, water, and first-aid supplies.

Case ID: 5 | Name: Low-Severity Urban Flood - Sample Case
  Disaster: Flood | Severity: Low | Location: Urban | Population: 800
  Evacuation: Monitor flood-prone streets and advise residents to move if conditions worsen.
  Shelter: Identify a nearby safe location and keep evacuation centers on standby.
  Resources: Prepare warning messages, drinking water, basic first aid, and transport if needed.

Case ID: 6 | Name: New Urban Typhoon
  Disaster: Typhoon | Severity: High | Location: Urban | Population: 5,000
  Evacuation: Prioritize urgent evacuation from high-risk areas. Coordinate with local emergency officials for approximately 5,000 potentially affected people. Prioritize low-lyi
ng streets, drainage bottlenecks, and densely populated areas.
  Shelter: Urgently confirm sufficient shelter capacity for up to 5,000 people, including accessible facilities and safe transport.
  Resources: Prioritize rescue teams, safe transport, drinking water, food, first-aid supplies, and reliable emergency communications. Monitor official typhoon, rainfall, and win
d advisories.
```



C. Reflection (5–7 sentences)

I developed the Disaster Response Decision Support System using Python. I created the sample disaster cases, implemented the similarity scoring method, and added the solution adaptation and case retention features. One challenge I encountered was making the program compare disaster cases and recommend an appropriate response plan. I addressed this by using weighted scores and conditions based on the disaster's characteristics. Through this activity, I learned how Case-Based Reasoning uses past experiences to solve new problems. I also understood how retaining new cases allows the system to build its knowledge for future situations.

Your Code - Laboratory Tasks

```
# Disaster Response Decision Support System using Case-Based Reasoning (CBR)
# CS0065 - Intelligent Systems | TA3
#

import json
from pathlib import Path

CASE_FILE = Path("disaster_cases.json")

# Step 1: Case Representation
# Each case stores the problem attributes and a previous response plan.
INITIAL_CASES = [
   {
       "case_id": 1,
       "name": "Typhoon Ursula - Sample Case",
       "problem": {
           "disaster_type": "typhoon",
           "flood_severity": "high",
           "location_type": "coastal",
           "affected_population": 3000
       },
       "solution": {
              "evacuation": "Evacuate residents in high-risk coastal and flood-prone
zones.",
           "shelter": "Prepare temporary shelters and confirm safe capacity.",
            "resources": "Prioritize rescue boats, food, drinking water, and first-aid
supplies."
```

```
        }
    },
    {
        "case_id": 2,
        "name": "Urban Flooding - Sample Case",
        "problem": {
            "disaster_type": "flood",
            "flood_severity": "high",
            "location_type": "urban",
            "affected_population": 5000
        },
        "solution": {
                "evacuation": "Move residents from deeply flooded streets and low-lying neighborhoods.",
                "shelter": "Open accessible evacuation centers and check their remaining capacity.",
                 "resources": "Prioritize rescue vehicles, clean water, food packs, and medical supplies."
        }
    },
    {
        "case_id": 3,
        "name": "Rural River Flood - Sample Case",
        "problem": {
            "disaster_type": "flood",
            "flood_severity": "moderate",
            "location_type": "rural",
            "affected_population": 1200
        },
        "solution": {
             "evacuation": "Move households near the river to designated safe areas when advised by authorities.",
            "shelter": "Prepare a nearby community shelter and transport assistance.",
                "resources": "Prepare drinking water, food packs, first-aid kits, and transport support."
        }
    },
    {
        "case_id": 4,
```

```
        "name": "Coastal Storm Surge - Sample Case",
        "problem": {
            "disaster_type": "typhoon",
            "flood_severity": "moderate",
            "location_type": "coastal",
            "affected_population": 2200
        },
        "solution": {
                    "evacuation": "Issue early warnings and move residents away from
storm-surge risk areas.",
            "shelter": "Prepare inland evacuation centers and transportation.",
             "resources": "Prepare communication equipment, food, water, and first-aid
supplies."
        }
    },
    {
        "case_id": 5,
        "name": "Low-Severity Urban Flood - Sample Case",
        "problem": {
            "disaster_type": "flood",
            "flood_severity": "low",
            "location_type": "urban",
            "affected_population": 800
        },
        "solution": {
              "evacuation": "Monitor flood-prone streets and advise residents to move if
conditions worsen.",
             "shelter": "Identify a nearby safe location and keep evacuation centers on
standby.",
              "resources": "Prepare warning messages, drinking water, basic first aid,
and transport if needed."
        }
    }
]

SEVERITY_LEVELS = {"low": 1, "moderate": 2, "high": 3}
LOCATION_OPTIONS = {"urban", "rural", "coastal"}
DISASTER_OPTIONS = {"typhoon", "flood", "earthquake"}
```

```
# Weighted similarity: disaster type 35%, flood severity 30%,
# location type 20%, and population similarity 15%.
WEIGHTS = {
    "disaster_type": 0.35,
    "flood_severity": 0.30,
    "location_type": 0.20,
    "affected_population": 0.15
}


def load_cases():
    """Load retained cases from JSON, or start with the five sample cases."""
    if CASE_FILE.exists():
        try:
            with CASE_FILE.open("r", encoding="utf-8") as file:
                saved_cases = json.load(file)
            if isinstance(saved_cases, list) and saved_cases:
                return saved_cases
        except (json.JSONDecodeError, OSError):
            print("Warning: Could not read saved case file. Using sample cases.")
    return [case.copy() for case in INITIAL_CASES]


def save_cases(cases):
    """Save the full case base so retained cases remain available next run."""
    with CASE_FILE.open("w", encoding="utf-8") as file:
        json.dump(cases, file, indent=4)


def get_nonempty_input(prompt):
    """Keep asking until the user enters a non-empty value."""
    while True:
        value = input(prompt).strip()
        if value:
            return value


def get_choice(prompt, choices):
    """Ask for a value from a fixed list of choices."""
```

```
    choices_text = "/".join(sorted(choices))
    while True:
        value = input(f"{prompt} ({choices_text}): ").strip().lower()
        if value in choices:
            return value
        print(f"Please enter one of these choices: {choices_text}")


def get_population():
    """Validate that the population is a positive whole number."""
    while True:
        try:
            population = int(input("Estimated affected population: ").strip())
            if population > 0:
                return population
            print("Population must be greater than zero.")
        except ValueError:
            print("Please enter a whole number, such as 2500.")


def collect_new_problem():
    """Collect the attributes of the new disaster case."""
    print("\n=== ENTER NEW DISASTER ===")
    name = get_nonempty_input("Case name (e.g., New Urban Typhoon): ")
    disaster_type = get_choice(
        "Disaster type", DISASTER_OPTIONS
    )
    flood_severity = get_choice(
        "Flood/impact severity", set(SEVERITY_LEVELS)
    )
    location_type = get_choice(
        "Location type", LOCATION_OPTIONS
    )
    affected_population = get_population()

    return {
        "case_id": None,
        "name": name,
        "problem": {
```

```
            "disaster_type": disaster_type,
            "flood_severity": flood_severity,
            "location_type": location_type,
            "affected_population": affected_population
        },
        "solution": {}
    }


def population_similarity(new_population, old_population):
    """Return a score from 0 to 1; closer population sizes score higher."""
    larger = max(new_population, old_population)
    difference = abs(new_population - old_population)
    return max(0.0, 1.0 - (difference / larger))


def calculate_similarity(new_case, old_case):
    """Calculate a weighted similarity score between 0 and 100."""
    new = new_case["problem"]
    old = old_case["problem"]

    scores = {
        "disaster_type": 1.0 if new["disaster_type"] == old["disaster_type"] else 0.0,
          "flood_severity": 1.0 if new["flood_severity"] == old["flood_severity"] else
0.0,
        "location_type": 1.0 if new["location_type"] == old["location_type"] else 0.0,
        "affected_population": population_similarity(
            new["affected_population"], old["affected_population"]
        )
    }

    weighted_score = sum(scores[key] * WEIGHTS[key] for key in WEIGHTS)
    return weighted_score * 100


def retrieve_similar_cases(new_case, cases):
    """Rank all previous cases from most similar to least similar."""
    ranked_cases = []
    for case in cases:
```

```
        score = calculate_similarity(new_case, case)
        ranked_cases.append((score, case))

    ranked_cases.sort(key=lambda item: item[0], reverse=True)
    return ranked_cases


def adapt_solution(new_case, retrieved_case):
    """Adapt the retrieved response plan using the new case's details."""
    problem = new_case["problem"]
    old_solution = retrieved_case["solution"]
    population = problem["affected_population"]
    severity = problem["flood_severity"]
    location = problem["location_type"]
    disaster = problem["disaster_type"]

    # Start with the retrieved case's solution, then tailor each recommendation.
    if severity == "high":
        evacuation = (
            f"Prioritize urgent evacuation from high-risk areas. Coordinate with "
              f"local emergency officials for approximately {population:,} potentially
affected people."
        )
        shelter = (
                f"Urgently confirm sufficient shelter capacity for up to {population:,}
people, "
            "including accessible facilities and safe transport."
        )
        resources = (
             "Prioritize rescue teams, safe transport, drinking water, food, first-aid
supplies, "
            "and reliable emergency communications."
        )
    elif severity == "moderate":
        evacuation = (
             f"Prepare targeted evacuation for high-risk areas and issue clear warnings
to "
                      f"communities within the potentially affected population of
{population:,}."
```

```
        )
        shelter = (
              f"Check nearby evacuation centers and available capacity for approximately
"
            f"{population:,} potentially affected people."
        )
        resources = (
                "Stage drinking water, food packs, first-aid supplies, transport, and
communication equipment."
        )
    else:
        evacuation = (
               "Monitor conditions, issue precautionary warnings, and prepare to move
residents "
            "if official alerts or local conditions worsen."
        )
        shelter = (
            f"Identify safe locations and keep shelter arrangements ready for up to "
            f"{population:,} potentially affected people."
        )
        resources = (
              "Prepare warning messages, basic first-aid supplies, drinking water, and
transport if needed."
        )

    if location == "coastal":
          evacuation += " Give particular attention to coastal and storm-surge risk
zones."
    elif location == "urban":
         evacuation += " Prioritize low-lying streets, drainage bottlenecks, and densely
populated areas."
    else:
          evacuation += " Pay attention to riverbanks, isolated communities, and access
routes."

    if disaster == "typhoon":
        resources += " Monitor official typhoon, rainfall, and wind advisories."
    elif disaster == "earthquake":
```

```
        resources += " Follow official earthquake safety guidance and check buildings
and routes for hazards."
    else:
        resources += " Monitor official rainfall, river-level, and flood advisories."

    return {
        "evacuation": evacuation,
        "shelter": shelter,
        "resources": resources,
        "adapted_from": retrieved_case["name"],
        "reference_solution": {
            "evacuation": old_solution.get("evacuation", ""),
            "shelter": old_solution.get("shelter", ""),
            "resources": old_solution.get("resources", "")
        }
    }


def display_case(case):
    """Print a case in a readable format."""
    problem = case["problem"]
    print(f"\nCase ID: {case['case_id']} | Name: {case['name']}")
    print(
        f"  Disaster: {problem['disaster_type'].title()} | "
        f"Severity: {problem['flood_severity'].title()} | "
        f"Location: {problem['location_type'].title()} | "
        f"Population: {problem['affected_population']:,}"
    )
    if case.get("solution"):
        solution = case["solution"]
        print(f"  Evacuation: {solution.get('evacuation', 'N/A')}")
        print(f"  Shelter: {solution.get('shelter', 'N/A')}")
        print(f"  Resources: {solution.get('resources', 'N/A')}")


def main():
    cases = load_cases()

    print("=" * 65)
```

```
print(" DISASTER RESPONSE DECISION SUPPORT SYSTEM (CBR)")
print("=" * 65)
print(f"Current case base: {len(cases)} case(s)")
print("\n=== EXISTING CASE BASE ===")
for case in cases:
    display_case(case)

new_case = collect_new_problem()

# Step 2: Similarity Assessment and Retrieval
ranked_cases = retrieve_similar_cases(new_case, cases)
print("\n=== SIMILARITY ASSESSMENT ===")
for rank, (score, case) in enumerate(ranked_cases, start=1):
    print(f"{rank}. {case['name']} - Similarity: {score:.2f}%")

best_score, best_case = ranked_cases[0]
print("\n=== RETRIEVED MOST SIMILAR CASE ===")
display_case(best_case)
print(f"Similarity score: {best_score:.2f}%")

# Step 3: Case Adaptation
print("\n=== ADAPTED RESPONSE PLAN ===")
adapted_solution = adapt_solution(new_case, best_case)
new_case["solution"] = adapted_solution
print(f"Adapted from: {adapted_solution['adapted_from']}")
print(f"1. Evacuation: {adapted_solution['evacuation']}")
print(f"2. Shelter: {adapted_solution['shelter']}")
print(f"3. Resources: {adapted_solution['resources']}")

# Step 4: Learning / Retention
new_case["case_id"] = max(
    (case.get("case_id", 0) for case in cases), default=0
) + 1
cases.append(new_case)
save_cases(cases)

print("\n=== LEARNING / RETENTION ===")
print("New case saved successfully!")
print(f"Saved to: {CASE_FILE.resolve()}")
```

```
    print(f"Updated case base contains {len(cases)} case(s).")

    print("\n=== UPDATED CASE BASE ===")
    for case in cases:
        display_case(case)

    print("\nReminder: This is a classroom prototype. Follow official emergency")
    print("alerts and local emergency-management authorities in real situations.")


if __name__ == "__main__":
    main()
```